

704. Binary Search **Easy**

Acceptance: 60.4% | [LeetCode](#)

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

```
**Input:** nums = [-1,0,3,5,9,12], target = 9
**Output:** 4
**Explanation:** 9 exists in nums and its index is 4
```

Example 2:

```
**Input:** nums = [-1,0,3,5,9,12], target = 2
**Output:** -1
**Explanation:** 2 does not exist in nums so return -1
```

Constraints:

- `1 <= nums.length <= 104`
- `-104 < nums[i], target < 104`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

Claude Opus Solution

704. Binary Search — Complete Solution Guide

1. Intuition

The **key insight** is that the array is **sorted**. Whenever you have a sorted collection and need to search for an element, you can leverage the ordering to **eliminate half of the search space** in every step.

Instead of checking each element one by one (which would be $O(n)$), we look at the **middle element**: - If it equals the target → found it! - If it's **less than** the target → the target must be on the **right half** (everything left is even smaller). - If it's **greater than** the target → the target must be on the **left half** (everything right is even larger).

By repeatedly halving the range, we reach the answer in **$O(\log n)$** steps. This is the classic **Binary Search** pattern.

2. Approach (Pseudocode)

- Set two pointers: `left = 0`, `right = len(nums) - 1`
- While `left <= right` (there is still a valid search window):
 - Compute `mid = (left + right) // 2`
 - If `nums[mid] == target` → return `mid`
 - If `nums[mid] < target` → discard left half: `left = mid + 1`
 - If `nums[mid] > target` → discard right half: `right = mid - 1`
- If loop ends, target not found → return `-1`

3. Python Solution

```
from typing import List
```

```
class Solution:
```

```
    def search(self, nums: List[int], target: int) -> int:
```

```
        # Define the search window using two pointers.
```

```
        # 'left' is the first index, 'right' is the last index.
```

```
        left, right = 0, len(nums) - 1
```

```
        # Continue while the window is valid (left has not crossed right).
```

```
        # We use '<=' because when left == right, there is still ONE element
```

```
        # left to inspect, and we must not skip it.
```

```
        while left <= right:
```

```
            # Compute the middle index.
```

```
            # Using (left + right) // 2 is fine in Python (no integer overflow),
```

```
            # but a safer pattern in other languages is: left + (right - left) // 2
```

```
            mid = (left + right) // 2
```

```
            if nums[mid] == target:
```

```
                # Found the target - return its index immediately.
```

```

        return mid
    elif nums[mid] < target:
        # The middle value is too small, so the target (if present)
        # must be in the RIGHT half. Move 'left' past 'mid'.
        left = mid + 1
    else: # nums[mid] > target
        # The middle value is too large, so the target (if present)
        # must be in the LEFT half. Move 'right' before 'mid'.
        right = mid - 1

# The window became empty without finding the target.
return -1

```

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(\log n)$	Each iteration discards half the remaining elements. Starting from n elements, the count shrinks: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$. The number of halvings is $\log_2(n)$.
Space	$O(1)$	We only use a constant number of variables (<code>left</code> , <code>right</code> , <code>mid</code>). The iterative version uses no extra memory that grows with input size.

5. Alternative Approaches

a) Recursive Binary Search

Same logic, but expressed with recursion. **Note:** this changes space complexity to $O(\log n)$ due to the recursion call stack.

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        def helper(left: int, right: int) -> int:
            if left > right: # base case: empty window
                return -1
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                return helper(mid + 1, right) # search right half
            else:
                return helper(left, mid - 1) # search left half

        return helper(0, len(nums) - 1)

```

b) Built-in bisect Module

Python's standard library provides binary search utilities. Useful in interviews to demonstrate knowledge, though most interviewers want the manual implementation.

```

import bisect

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        # bisect_left finds the leftmost insertion point for target.
        i = bisect.bisect_left(nums, target)
        # Verify the position is valid AND actually contains the target.
        if i < len(nums) and nums[i] == target:
            return i
        return -1

```

⚠ Common Pitfalls to Remember

- left <= right vs left < right** — Use `<=` here. With `<`, you'd miss checking the case when `left == right` (a single-element window).
- mid + 1 / mid - 1** — Always move past `mid` to avoid **infinite loops** (e.g., if you set `left = mid` when `nums[mid] < target`, the pointers may never advance).
- Overflow** — Not an issue in Python, but in C++/Java prefer `left + (right - left) // 2` to avoid integer overflow.

The iterative version is the **recommended interview answer**: clean, $O(1)$ space, and no recursion overhead.

Codex Solution

1. Intuition

Because `nums` is already sorted in ascending order, we do **not** need to scan every element.

Instead, we can repeatedly check the middle element:

- If `nums[mid] == target`, we found the answer.
- If `nums[mid] < target`, then the target must be on the **right side**.
- If `nums[mid] > target`, then the target must be on the **left side**.

Each step eliminates half of the remaining search space, which gives us $O(\log n)$ time complexity.

This is the classic **Binary Search** pattern.

2. Approach

Use two pointers:

- `left` starts at the beginning of the array.
- `right` starts at the end of the array.

Pseudocode

```
left = 0
right = len(nums) - 1

while left <= right:
    mid = middle index between left and right

    if nums[mid] == target:
        return mid

    else if nums[mid] < target:
        left = mid + 1

    else:
        right = mid - 1

return -1
```

3. Python Solution

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        # Initialize the search boundaries.
        # left points to the beginning of the current search range.
        # right points to the end of the current search range.
        left, right = 0, len(nums) - 1

        # Continue searching while the range is valid.
        # If left becomes greater than right, the target is not present.
        while left <= right:
            # Find the middle index of the current search range.
            # In Python, overflow is not an issue, but this is the standard formula.
            mid = (left + right) // 2

            # Case 1: We found the target.
            if nums[mid] == target:
                return mid

            # Case 2: Middle value is smaller than target.
            # Since the array is sorted, everything to the left of mid
```

```
# is also smaller than target, so search the right half.
elif nums[mid] < target:
    left = mid + 1

# Case 3: Middle value is greater than target.
# Since the array is sorted, everything to the right of mid
# is also greater than target, so search the left half.
else:
    right = mid - 1

# If we exit the loop, the target does not exist in nums.
return -1
```

4. Complexity Analysis

Let n be the length of `nums`.

Time Complexity: $O(\log n)$

At each step, binary search cuts the search space in half.

For example:

```
n -> n/2 -> n/4 -> n/8 -> ...
```

So the number of operations grows logarithmically.

Space Complexity: $O(1)$

We only use a few variables:

- `left`
- `right`
- `mid`

No extra data structures are used.

5. Alternative Approaches

1. Linear Search

We could scan every element from left to right:

```
for i in range(len(nums)):
    if nums[i] == target:
        return i
return -1
```

However, this takes $O(n)$ time, which does not satisfy the required $O(\log n)$ runtime.

2. Built-in Search Methods

Python has methods like `list.index()`, but they also perform linear search internally, so they are $O(n)$.

3. Recursive Binary Search

Binary search can also be implemented recursively, but the iterative version is usually preferred because it uses $O(1)$ space instead of $O(\log n)$ recursive call stack space.